Assignment 1 – Fundamental R Programming and Gene Data Analysis

Mohamed Hussein

2025-08-09

Objective

This R notebook demonstrates foundational R programming concepts such as vector creation, data subsetting, basic file handling, and simple data manipulation.
The goal is to practice essential R operations including indexing, sequencing, descriptive analysis, and vectorized computations using gene-related data.

Workflow Overview: 1. Create and manipulate vectors.
2. Subset and transform elements in numeric sequences.
3. Explore files in the working directory.
4. Perform simple bioinformatics-style data operations on gene expression and gene length. 5. Compute derived metrics such as normalized gene expression.

Exercise 1 – Creating Vectors

1. Create a vector containing the values 1,2,3,4,5

```r
vec1 <- c(1,2,3,4,5)      # Create a vector with elements 1-5
print(vec1)               # Display the vector
```

```
## [1] 1 2 3 4 5
```

2. Create a vector containing the values 1 to 100

```r
vec2 <- 1:100             # Generate a sequence from 1 to 100
print(vec2)               # Display the sequence
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100
```

3. Create a vector containing the values 0,5,10,15,20

```r
vec3 <- seq(0, 20, by=5) # Create a sequence with step size 5
print(vec3)              # Display the sequence
```

```
## [1]  0  5 10 15 20
```

4. Create a vector containing the values 1,1,2,2,3,3

```r
vec4 <- c(1,1,2,2,3,3)   # Create a repeated pattern vector
print(vec4)              # Display the vector
```

```
## [1] 1 1 2 2 3 3
```

5. Create a vector containing the values 1,1,5,7,9,10

```r
vec5 <- c(1,1,5,7,9,10)  # Create a custom numeric vector
print(vec5)              # Display the vector
```

```
## [1]  1  1  5  7  9 10
```

Exercise 2 – Subsetting Vectors

1. Create a vector containing the values 1 to 10

```r
v <- 1:10                # Create a numeric sequence from 1 to 10
print(v)                 # Display the vector
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10
```

2. Create a new vector with all but the first and last value

```r
v2 <- v[-c(1,length(v))] # Remove first and last elements
print(v2)                # Display the new vector
```

```
## [1] 2 3 4 5 6 7 8 9
```

3. Create a new vector with all but the second and fifth value

```r
v3 <- v[-c(2,5)]          # Remove elements at positions 2 and 5
print(v3)                 # Display the new vector
```

```
## [1]  1  3  4  6  7  8  9 10
```

4. Create a new vector of square root of the sixth and seventh position

```r
v4 <- sqrt(v[c(6,7)])     # Compute square roots of positions 6 and 7
print(v4)                 # Display the results
```

```
## [1] 2.449490 2.645751
```

5. Create a new vector of alternating positions in the vector using another vector

```r
indices <- seq(1, length(v), by=2) # Create sequence of odd indices
v5 <- v[indices]                   # Select alternating elements
print(v5)                          # Display the alternating elements
```

```
## [1] 1 3 5 7 9
```

Exercise 3 – Working with Files

1. Find the number of files in the present working directory

```r
num_files <- length(list.files())  # Count files in working directory
print(num_files)                   # Display the count
```

```
## [1] 2
```

2. List the first file in the present working directory

```r
first_file <- list.files()[1]      # Get the first file name
print(first_file)                  # Display the first file
```

```
## [1] "Assignment1_Fundamental_R_Programming_and_Gene_Data_Analysis.pdf"
```

Exercise 4 – Gene Expression Data

1. Create a vector of the gene names Gene_1, Gene_2, Gene_3, Gene_4

```r
genes <- c("Gene_1", "Gene_2", "Gene_3", "Gene_4")  # Define gene names
```

2. Create a vector of the expression values 1000, 3000, 10000, 12000

```r
expression <- c(1000, 3000, 10000, 12000)            # Define expression values
```

3. Create a vector of the gene lengths 100, 3000, 200, 1000

```r
lengths <- c(100, 3000, 200, 1000)                   # Define gene lengths
```

4. Assign the gene names as header for the expression vector

```r
names(expression) <- genes   # Assign gene names as names of expression vector
print(expression)            # Display named expression vector
```

```
## Gene_1 Gene_2 Gene_3 Gene_4
##   1000   3000  10000  12000
```

5. Assign the gene names as header for the gene lengths vector

```r
names(lengths) <- genes      # Assign gene names as names of lengths vector
print(lengths)               # Display named lengths vector
```

```
## Gene_1 Gene_2 Gene_3 Gene_4
##    100   3000    200   1000
```

6. Find the longest gene

```r
longest_gene <- genes[which.max(lengths)]  # Identify gene with maximum length
print(longest_gene)                        # Display longest gene
```

```
## [1] "Gene_2"
```

7. Identify genes which have length > 100 and expression > 10000

```r
genes_selected <- genes[(lengths > 100) & (expression > 10000)]  # Filter by conditions
print(genes_selected)                                            # Display selected genes
```

```
## [1] "Gene_4"
```

Additonal Questions – Derived Calculations

1. Create a vector for gene copy number

```r
copy_number <- c(10, 1, 50, 12)   # Define gene copy numbers
names(copy_number) <- genes       # Assign gene names as headers
```

2. Calculate expression over gene length (length-normalized expression)

```r
length_norm_expr <- expression / lengths  # Compute normalized expression
print(length_norm_expr)                   # Display normalized expression
```

```
## Gene_1 Gene_2 Gene_3 Gene_4
##     10      1     50     12
```

3. Identify genes with length-normalized expression greater than average

```r
genes_high_norm_expr <- genes[length_norm_expr > mean(length_norm_expr)]  # Filter genes above average
print(genes_high_norm_expr)                                               # Display results
```

```
## [1] "Gene_3"
```